

Custom Compiler™ iPDK Developer Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	5
Related Products, Publications, and Trademarks	5
Conventions	5
Customer Support	6
Statement on Inclusivity and Diversity	7

1. Introduction to iPDK Development	8
What is a PDK?	8
What is an iPDK?	9
Structure of an iPDK	9
iPDK Development Prerequisites	10
iPDK Development Stages	10

2. The Technology File, Display Resource File, and Mapping File	12
Technology File	12
Contents of a Technology File	12
Rules for Creating a Technology File	13
Creating a Technology File	14
Display Resource File	14
Contents of a Display Resource File	15
Rules for Creating a Display Resource File	15
Creating a Display Resource File	16
Mapping File	16
Rules for Creating a Mapping File	17
Creating a Mapping File	17

3. Creating Front-End Components	18
Create a Library	18
Copy Symbols From the analogLib Into your Library	19
Develop the Core Front-End Components	20

Parameter Definitions	21
Label Display and Netlist Info Section	23
Property Bag Definitions	26
Write and Assign Callback Procedures	27
Copy Simulation Views From the analogLib Into your Library	30
4. Creating Back-End Components	31
Creating Interoperable PCells Using the Python Language	31
Writing the PyCell Code	32
Creating the __init__.py File	34
Creating the PyCell SuperMaster	34
Creating ivpcell Views	35
5. Preparing for Physical Verification and Extraction	36
Physical Verification and Extraction Prerequisites	36
Rules for Creating Device Mapping File and Output Layer Mapping File	37
Create Device Mapping File	37
Create Output Layer Mapping File	37
6. Qualifying and Packaging the iPDK	38
Qualify the iPDK	38
Package the iPDK	38
A. Common and Advanced Operations	40
PyCell Studio for Developing PyCells	40
Using the Custom Compiler Parameter Definition Editor Tool	40
Adding iCDF Parameters	40
Adding simInfo	42
Glossary	44

About This Manual

This manual explains how to develop, qualify, and package a new Interoperable Process Design Kit (iPDK).

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® Custom Compiler™

Synopsys® IC Validator

Synopsys® PyCell Studio

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
<code>Courier</code>	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to iPDK Development

Describes an Interoperable Process Design Kit (iPDK), its contents, and the process for developing an iPDK.

This section covers the following topics:

- [What is a PDK?](#)
- [What is an iPDK?](#)
- [Structure of an iPDK](#)
- [iPDK Development Prerequisites](#)
- [iPDK Development Stages](#)

What is a PDK?

A Process Design Kit (PDK) is a set of files, typically developed by an IC chip manufacturer (foundry or IDM), that describes the process specific information that must be used by EDA tools for the design of integrated circuits of all types. PDKs are particularly important in the design of analog/mixed signal integrated circuits and the development of digital design libraries.

A PDK contains the information needed by EDA tools to assist in the analog mixed signal design process. Typically this information is provided by silicon foundries and is specific to a particular process at a particular technology node. Typically the PDK includes schematic symbols, SPICE models, rule decks with the technology that allows designs to be validated, parameterized cells, a list of parameters that specify a cell along with default values for the parameters, and callbacks or evaluator code that must be run when one or more of these parameters change.

What is an iPDK?

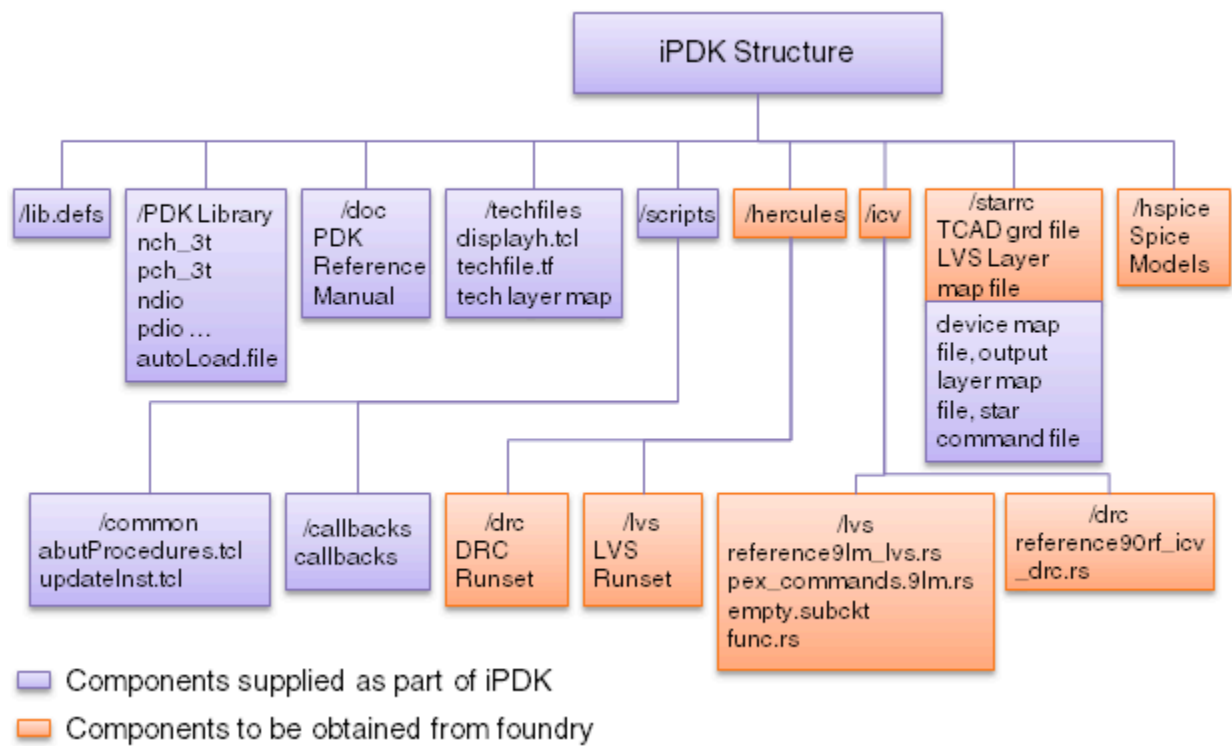
The need for developing an Interoperable Process Design Kit (iPDK) was due to the fact that the PDKs were pre-qualified only against specific EDA tools and not against a set of common EDA tools available in the industry.

The iPDK addresses this need and has been extensively tested among an array of EDA vendor tools. The iPDK provides the following advantages:

- One PDK for all custom tools, thereby reducing the PDK development and maintenance turnaround time
- Choice of tools for front-to-back Analog flow
- Open database (OpenAccess) for PDK development
- No necessity for data translation
- State-of-the-art Parameterized Cell (PCell) development
- Improved PCell reuse across technology nodes

Structure of an iPDK

The following diagram is an illustration of a standard iPDK structure:



iPDK Development Prerequisites

Following are the prerequisites for developing an iPDK:

- Understanding of the purpose and organization of a PDK
- Exposure to working with OpenAccess database
- Exposure to coding in Tcl language
- Exposure to coding in Python language

iPDK Development Stages

The following table describes the different stages in an iPDK development workflow:

Stage No.	Stage	Stage Description
1	The Technology File, Display Resource File, and Mapping File	• Creating a Technology File (<code>.tf</code>) • Creating a Display Resource File (<code>.tcl</code>) • Creating a Mapping File

Stage No.	Stage	Stage Description
2	Creating Front-End Components	• Creating a Library • Creating Symbols for Schematic • Defining Simulation Information (simInfo) • Defining Interoperable Component Design Format (iCDF) Parameters • Defining Property Bags • Writing and Assigning Tcl Callback Scripts • Creating Simulation Views
3	Creating Back-End Components	• Creating PyCells • Creating ivpcell Views
4	Preparing for Physical Verification and Extraction	• DRC Runset File • LVS Runset File • Device-Map file • Output Layer-Map file
5	Qualifying and Packaging the iPDK	• iPDK verification • iPDK packaging

2

The Technology File, Display Resource File, and Mapping File

Describes the purpose, structure, and procedure for creating a technology file, a display resources file and a mapping file.

A technology file provides technology-specific information, such as layer names, physical characteristics of each metal layer, and the routing design rules in ASCII format.

The section covers the following topics:

- [Technology File](#)
- [Display Resource File](#)
- [Mapping File](#)

Technology File

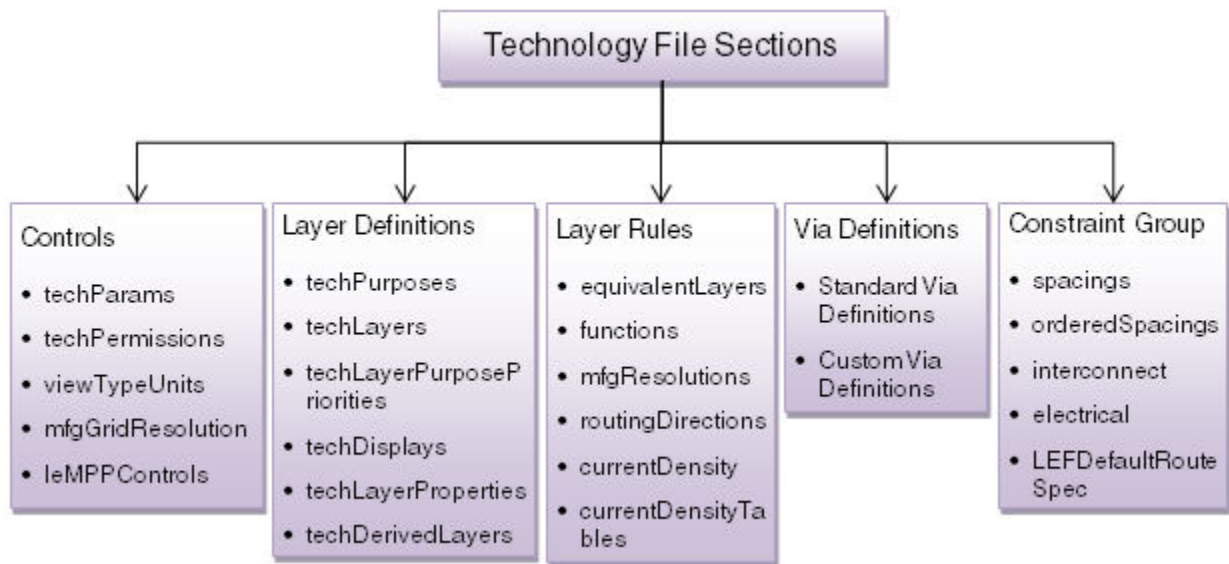
The technology file is used for adding technology information to a library. The technology file typically contains layer information, design rule definitions, and associations with display packets defined in the display resource file.

This section covers the following topics:

- [Contents of a Technology File](#)
- [Rules for Creating a Technology File](#)
- [Creating a Technology File](#)

Contents of a Technology File

The technology file must contain the following sections and subsections in the order in which they are listed here:



Rules for Creating a Technology File

The following rules must be followed while creating a technology file:

- Each section or a subsection must begin with a keyword followed by an open parentheses "(" and end with a close parentheses ")".
- A section or a subsection in the technology file can contain more than one attribute statement.
- Each attribute statement within a section may consist of a sequence of attributes, which must be enclosed within parentheses.
- Each attribute in an attribute statement must be separated with a space.

- Lines that should be treated as a comment must be preceded by a semicolon (";").

```
controls(
  techParams(
    ; ( parameter      value )
    ; ( -----      -)
    ( maskGrid        0.005 )
    ( cadGrid          0.005 )
    ( drcGrid          0.005 )
    ( mfgGrid          0.005 )
    ( scale            1.0 )
    ( MPPS              (list (list nil (quote cwW) 0.09
                                (quote cwL) 0.09
                                (quote cwSp) 0.10)))

  ) ; techParams
)
```

Diagram labels with arrows pointing to the code:

- Section: points to `controls(`
- Subsection: points to `techParams(`
- Comment line: points to `; ( parameter value )`
- Attributes: points to `( maskGrid 0.005 )`

Creating a Technology File

For a detailed explanation about defining the technology data in each of the sections and subsections, see the Technology and Display Resource Files section in the *Custom Compiler Technology File and Display Resource Reference Manual*.

Display Resource File

The purpose of the display resource file is to control how the layers and layer purpose pairs (LPPs) are displayed in the layout tool. The display resource file contains the colors, fills, stipples, and packets information for the layers and LPPs.

This section covers the following topics:

- [Contents of a Display Resource File](#)
- [Rules for Creating a Display Resource File](#)
- [Creating a Display Resource File](#)

Contents of a Display Resource File

The display resource file must contain the following sections in the order in which they are listed here:

1. Display Resource Header
2. Colors
3. Stipples
4. Line Styles
5. Display Packets
6. Display Packet Aliases
7. Text Packets

Rules for Creating a Display Resource File

The following rules must be followed while creating a display resource file:

- Each type of attribute must appear on a new line.
- Each attribute in an attribute statement must be separated with a space.

- Lines that should be treated as a comment must be preceded by a semicolon (";").

```
#DisplayResourceTcl ← Header
set display_object [db::getNext [dr::getDisplays -filter "%name = {display}"]]
if {$display_object = ""} {
    set display_object [dr::createDisplay display -maxColors 256
                                                -maxStipples 256
                                                -maxLineStyles 256]
}
# Colors definition ← Comment line
dr::createColor white -display $display_object -red 255 -green 255 -blue 255
# Stipples definition
dr::createStipple dots -display $display_object -pattern {
    {0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }

# LineStyles definition
dr::createLineStyle dashed -display $display_object -width 1
                                                -pattern [list 1 1 1 1 0 0]

# Packets definition
dr::createPacket defaultPacket -display $display_object -stipple blank \
                                -lineStyle solid -fill white -outline yellow

# PacketAliases definition
dr::createPacketAlias stroke -packet stroke_white -display $display_object
```

Attributes

Creating a Display Resource File

For a detailed explanation about defining the display attributes, see the Display Resource Files section in the *Custom Compiler Technology File and Display Resource Reference Manual*.

Mapping File

The purpose of the mapping file is to map the OpenAccess layer purpose pairs (LPPs) to GDSII layer numbers and data types. Map files are useful when importing and exporting designs in GDSII format.

This section covers the following topics:

- [Rules for Creating a Mapping File](#)
- [Creating a Mapping File](#)

Rules for Creating a Mapping File

The following rules must be followed while creating a mapping file:

- Each mapping relation must appear on a new line.
- Each attribute in a mapping relation must be separated with a space.
- Lines that should be treated as a comment must be preceded by a pound sign ("#").

#layerName	layerPurpose	layerNo.	layerDataType	← Comment line
nwell	drawing	1	0	
diff	drawing	2	0	
pplus	drawing	4	0	
nplus	drawing	5	0	
rpoly	drawing	6	0	
cont	drawing	7	0	

□ Rows: Mapping Relation

□ Column: Attributes

Creating a Mapping File

The mapping file must contain the following data columns in a table-like structure, where each row defines the relationship between the technology file layer and its purpose to the corresponding GDSII layer number and data type:

```
#layer_name    layerPurpose    layerNo    layerDataType
layer1         layerPurpose1   layerNo1   layerDataType1
layer2         layerPurpose2   layerNo2   layerDataType2
...
```

3

Creating Front-End Components

Describes how to create front-end components of an Interoperable Process Design Kit (iPDK).

The front-end components are the initial building blocks of an iPDK:

- A library, which will contain a collection of schematic symbols.
- Interoperable component description format (iCDF) parameters
- Simulation-specific information
- Property bag information
- Simulation views
- Interoperable callback procedures

This section covers the following topics:

- [Create a Library](#)
- [Copy Symbols From the analogLib Into your Library](#)
- [Develop the Core Front-End Components](#)
- [Write and Assign Callback Procedures](#)
- [Copy Simulation Views From the analogLib Into your Library](#)

Create a Library

Create a library to define a logical collection of data representing a complete design or a portion of it. A library is the top organizational level of design data.

To create a new library:

1. From the Custom Compiler Console menu, choose **File > New > Library**. Alternatively, click  from the toolbar. The New Library dialog box is displayed.
2. Enter the following information and click **OK**:
 - a. library name
 - b. location where the library must be created.
 - c. technology data that this library must refer to.

A new library with the specified name is created and is populated in the **Libraries** list in the Custom Compiler Library Manager window.

Copy Symbols From the analogLib Into your Library

After you are done creating the library, you need to add symbols to the library.

analogLib is a standard symbols library from Synopsys that is packaged with the Custom Compiler tool. You can use the symbols in your schematic designs and in the Custom Compiler simulation environment.

To copy symbols from the analogLib library into your library:

1. From the Custom Compiler Console menu, choose **Tools > Library Manager**.
Alternatively, click  from the toolbar. The Library Manager window is displayed.
2. In the Custom Compiler Library Manager window, select **analogLib** from the **Libraries** list, and select the cells you want to copy from the **Cells** list.

Note:

To select more than one cell for copying, press and hold the **Ctrl** key, and select the required cells.

To select a block of cells for copying, select the starting cell from the **Cells** list, press and hold the **Shift** key, and select the ending cell.

To select all cells for copying, select a cell from the **Cells** list and press **Ctrl +A**.

3. From the Custom Compiler Library Manager menu, choose **Edit > Copy > Cell**.

Alternatively, click  from the toolbar. The Copy Cell dialog box is displayed.

4. Select the **myLib** library from the **Libraries** list in the **Destination** group into which you want to copy the cells, and click **OK**. The selected cells from the analogLib library is copied into your library.

To create custom symbols, see Creating Symbols in the *Custom CompilerSchematic Editor User Guide*.

See Library Manager Tool in the *Custom Compiler Environment User Guide* for more information about copying symbols from one library to another.

Develop the Core Front-End Components

After copying the cells from the analogLib library into your library, you need to define the following components:

1. **Parameter Definitions**: Defines Interoperable Component Description Format (iCDF) Parameters
2. **Label Display and Netlist Info Section**: Defines simulator-specific simulation information (siminfo)
3. **Property Bag Definitions**: Defines the design rules that are not part of the technology file

Note:

The iCDF parameters, simInfo, and property bag information can be defined for either a library or for the cells in the library.

After you define the iCDFs, simInfo, property bags, you must source these files from the Custom Compiler console window. The Custom Compiler tool executes these files and loads the data into the OpenAccess database (`data.dm`).

Syntax for Library-Level Definitions:

```
set dmd [oa::LibDMDDataOpen $<libName> <mode w|r|a>]
oa::getAccess [oa::LibFind $libName] write 1
# Parameter definitions
...
# Label Display and Netlist Info Section
...
oa::save $dmd
oa::close $dmd
oa::releaseAccess [oa::LibFind $libName]
```

Syntax for Cell-Level Definitions:

```
set dmd [oa::CellDMDDataOpen $<libName> $<cellName> <mode w|r|a>]
oa::getAccess [oa::LibFind $libName] write 1
# Parameter definitions
...
# Label Display and Netlist Info Section
...
oa::save $dmd
oa::close $dmd
oa::releaseAccess [oa::LibFind $libName]
```

Parameter Definitions

This section contains a list of associative arrays. Each array defines one Interoperable Component Description Format (iCDF) parameter.

Note:

The CDFs should not have any vendor tool specific commands.

Syntax:

```
# Parameter definitions
db::createParamDef <string>
    -data <dmCell|oaCell|oaCellDMDData|
          oaLib|oaLibDMDData|oaStdViaDef|
          oaTech> \
    [-prompt <string>] \
    [-defValue <string>] \
    [-type <enum>] \
    [-callback <string>] \
    [-units <string>] \
    [-choices <string>] \
    [-parseAsNumber <boolean>] \
    [-parseAsPEL <boolean>] \
    [-editable <string>] \
    [-display <string>] \
    [-use <string>] \
    [-storeDefault <boolean>] \
    [-forceMaster <boolean>]
```

Argument	Description
string	Name of the iCDF.
	Note: The <i>IxComponentType</i> iCDF can be specified only at the library level. This iCDF is used for auto-chaining and auto-folding operations in SDL.
-data	Cell or library that contains the design manager data where the specified Netlist Info is stored.

Argument	Description
<code>-prompt</code>	Identifier used in the Schematic Editor to identify the parameter.
<code>-defValue</code>	Default value of the parameter. If no property exists on the instance, then this is the parameter value.
<code>-type</code>	Data type the Parameter Definition Editor uses to manipulate the parameter value. The default data type is "string". Valid data types are: • bool boolean • button • cyclic • filePath • float • int integer • lpp • netSet • radio • real • string
<code>-callback</code>	Callback for the parameter. This field is evaluated whenever an instance parameter using this definition is modified.
<code>-units</code>	International System of Units (SI) of measurement for the parameter.
<code>-choices</code>	List of choices to use for the enumeration parameter types.
<code>-parseAsNumber</code>	Determines whether the result of the evaluation should be treated as a number or as a string literal. Default is "true".
<code>-parseAsPEL</code>	Controls whether instance parameters should use the PEL expression evaluator. Default is "false".
<code>-editable</code>	This field indicates if the Property Editor allows direct editing of parameter instance values. This field is typically used when the value of the parameter is meant to be set by another parameter's change callback. Default is "t".
<code>-display</code>	Sets the default visibility of the parameter. Default is "t".
<code>-use</code>	This field combined with the "display" field specifies if the parameter should be displayed in the Property Editor. Default is "t".
<code>-storeDefault</code>	Upon instantiation, copies the <code>defValue</code> as a property on the instance. Default is "false".
<code>-forceMaster</code>	Forces the create operation to create the iCDF in the master library and ignore any overlay library. Default is "false".

Example 1 Library-Level iCDF

```
db::createParamDef lxComponentType \
  -data            $cmd \
  -prompt          {Custom Compiler Component Types} \
```

```
-defValue      {(("cells" ("n_3t" "na_3t" "nl_3t"
                          "nh_3t" "nm_3t" "n_4t"
                          "na_4t" "nl_4t" "nh_4t"
                          "nm_4t" "n25_3t" "na25_3t"
                          "n18_3t" "n33_3t" "na18_3t"
                          "na33_3t" "nm25_3t" "na25_4t"
                          "n18_4t" "n33_4t" "na18_4t"
                          "na33_4t" "nm25_4t"))
               ("type" "nmos")
               ("lxActiveLayer" "diff drawing")
               ("lxMOSDeviceType" "NMOS")
               ("lxDeviceWidth" "w")
               ("lxMaxWidth" 5u))}
-type          string
-parseAsNumber no
-parseAsPEL    no
-display       {nil}
```

Example 2 Cell-Level iCDF

```
db::createParamDef model \
  -data $data \
  -prompt {Spice Model} \
  -defValue {N} \
  -type string \
  -parseAsNumber 0 \
  -parseAsPEL 1 \
  -editable {nil} \
  -display {t} \
  -use {t} \
  -storeDefault 0
```

Label Display and Netlist Info Section

The Label Display Section (propList) and the Netlist info section of the Interoperable Component Description Format (iCDF) file contain settings for interpreted labels and also contain the Simulation Information (simInfo) subsection. The interpreted label settings control how `cdsParam()` and `cdsName()` labels are displayed in the Schematic Editor.

Syntax:

```
# Label display section
db::setNetlistInfo <string> \
  -data <dmCell|oaCell|oaCellDMDData| \
      oaLib|oaLibDMDData> \
  [-netlist <string>] \
  [-info <string>] \
  -value <string> \
  [-type <enum>] \
  [-section <enum>]
```

Argument	Description
string	Name of the netlist info property. Note: The <i>lxCComponentType</i> iCDF can be specified only at the library level. This iCDF is used for auto-chaining and auto-folding operations in SDL.
-data	Cell or library that contains the design manager data where the specified Netlist Info is stored.
-netlister	Name of the netlister,
-info	Netlist information. Default is "simInfo",
-value	Value to be associated with the specified netlist property.
-type	Data type the Parameter Definition Editor uses to manipulate the parameter value. The default data type is "symbol". Valid data types are: • floating • integer • list • real • string • symbol
-section	Name of the iCDF section. The default is "netlist". Valid values are: • display • form • netlist See Table 1 for a detailed explanation of the -section attributes.
-clear	When "true", clears the specified netlist property. This is useful before creating a new property with the -value argument. Default is "false".
-forceMaster	Forces the create operation to create the iCDF in the master library and ignore any overlay library. Default is "false".

Value	Attribute	Description
<code>display</code>	<code>instNameType</code>	This field can either be set to “schematic”, or “simulation”. If the value is set to “schematic” then <code>cdsName()</code> evaluates to the name of the instance within the schematic database. If the value is set to “simulation” then <code>cdsName()</code> evaluates to the name of the instance within the simulation netlist.

Table 1 -section Attributes (Continued)

Value	Attribute	Description
	<code>initDisplayMode</code>	This field can be set to either "cellName" or "instanceName". If the field is set to "cellName" then <code>cdsName()</code> labels are displayed as the name of the component. If the field is set to "instanceName" then the instance's name attribute is displayed in <code>cdsName()</code> labels.
	<code>modelLabelSet</code>	If the Interpreted Label Mode is set to "model" then this list specifies the simulator computed device model values to display with <code>cdsParam()</code> .
	<code>opPointLabelSet</code>	If the Interpreted Label Mode is set to "oppoint" then this list specifies the operating point values to display with <code>cdsParam()</code> .
	<code>paramLabelSet</code>	If the Interpreted Label Mode is set to "param" then this list specifies the instance parameter values to display with <code>cdsParam()</code> .
form	<code>promptWidth</code>	Controls the width of the prompt column in the property editor form.
	<code>fieldHeight</code>	Controls the height of the user input field in the property editor form.
	<code>fieldWidth</code>	Control the width of the user input field in the property editor form.
	<code>buttonFieldWidth</code>	Control the width of buttons associated with the display of parameters.
	<code>formInitProc</code>	Used to register a callback to be triggered when the property editor commands are executed.
	<code>doneProc</code>	Used to register a callback to be triggered when you click the OK or Apply button in the property editor.
netlist	<code>componentName</code>	The Custom Compiler netlist formatter uses the value of this field to determine how to netlist the instance(s).
	<code>instParameters</code>	Used by the netlist formatter for properties that should be written as "name = value" pairs.
	<code>modelName</code>	A variant of the model name to print. This takes precedence over the <code>componentName</code> .
	<code>namePrefix</code>	Used to specify the valid first character for this component.
	<code>netlistProcedure</code>	Used to determine the formatting procedure to call to write instance lines for this component.
	<code>otherParameters</code>	Used by Spice netlist formatter for properties that should be written as an ordered list as opposed to "name = value" pairs.

Table 1 -section Attributes (Continued)

Value	Attribute	Description
	<code>permuteRule</code>	Used by the Custom Compiler tool to check what terminals may be permuted during modification.
	<code>propMapping</code>	Used to specify a mapping from component parameter names to simulator-specific property names.
	<code>stringParameters</code>	Used to determine the parameters that support string values.
	<code>termMapping</code>	Used to specify the mapping of terminals for a simulator.
	<code>termOrder</code>	Used to specify the order of terminals to be used in an instance line.

Property Bag Definitions

Property bags are used for storing design rules that are not part of the technology file.

Typically common rules such as contact width and poly overlap of diffusion are defined in the library-level property bags and cell-specific rules such as min, max, and contact to gate spacing are defined in the cell-level property bags.

Property bags are accessed by both the callback procedures and the PCell codes. The callback procedures use the property bags for verifying and validating user inputs, while the PCell codes use the property bags to fetch design rule values.

Syntax:

```
# Property Bags
oa:: $<object> <name> <value>
```

Argument	Description
<code>variant</code>	Type of the property bag. "StringPropCreate" is the most commonly used property bag declaration. Some of the frequently used property bag declarations are: • StringPropCreate • DoublePropCreate • IntPropCreate • BooleanPropCreate
<code>object</code>	Name of the OpenAccess database.
<code>name</code>	Name of the property.
<code>value</code>	Value of the property.

Example 3 *Property Bag*

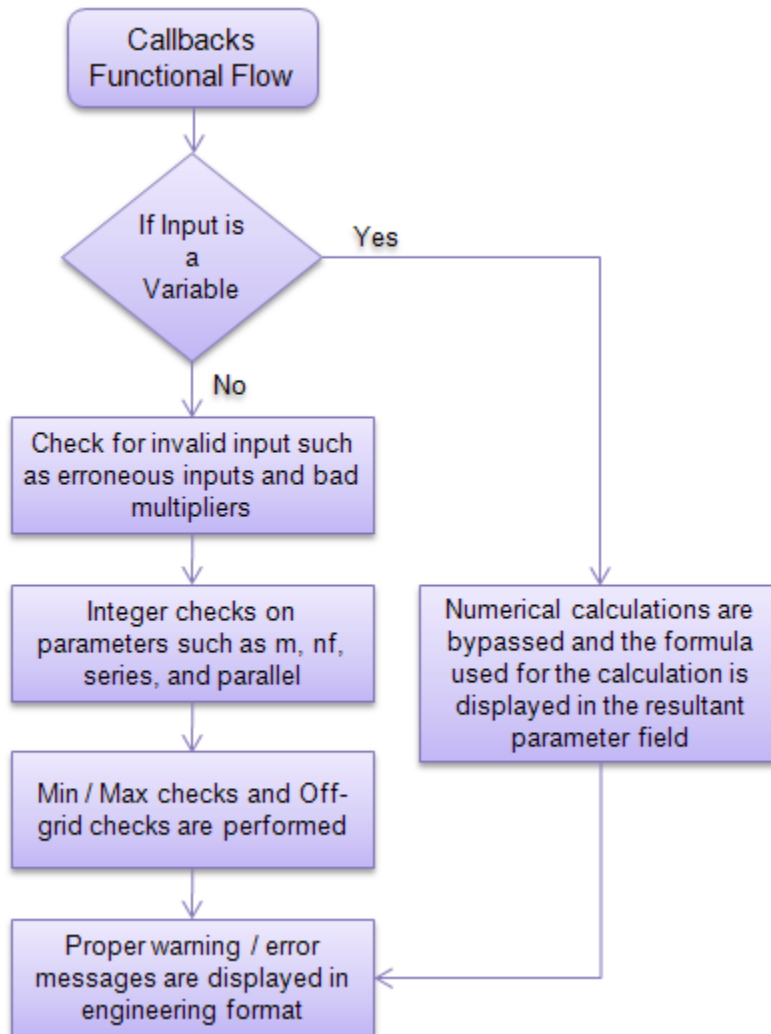
```
oa::StringPropCreate $dmd dWidthEL {480n}  
oa::DoublePropCreate $dmd MlCwEnc 0.005  
oa::IntPropCreate $dmd nf 3  
oa::DoublePropCreate $dmd DFM t
```

Write and Assign Callback Procedures

Callbacks are procedures that are executed when an iCDF parameter is changed. The callback procedures are used to verify and validate user inputs, calculate dependent parameters, and check parameter ranges.

Vendor-specific commands must be captured within the wrapper procedures that are prefixed with “iPDK_” and each vendor must develop these procedures for their specific tools.

The callback functional flow is as follows:



The following is an example of an interoperable callback procedure for a diode:

```

proc diode_cbk {} {
  set inst [db::getCurrentRef]
  set param [db::getCurrentParam]
  set len [iPDK_engToSci [iPDK_getParamValue l $inst]]
  set wid [iPDK_engToSci [iPDK_getParamValue w $inst]]
  set minWidth 0.1e-6
  set maxWidth 100e-6
  set minLength 0.1e-6
  set maxLength 100e-6
  set peri [expr 2*($len + $wid)]
  set peri [iPDK_sciToEng $peri]
  iPDK_setParamValue pj $peri $inst 0
  if {$wid < $minWidth} {
    puts "width value $wid is less than the recommended Min Width $minWidth"
    set widmin [iPDK_sciToEng $minWidth]
    iPDK_setParamValue w $widmin $inst 0
  }
}
  
```

```
if {$wid > $maxWidth} {  
    puts "width value $wid is greater than the recommended Max Width  
$maxWidth"  
    set widmax [iPDK_sciToEng $maxWidth]  
    iPDK_setParamValue w $widmax $inst 0  
}  
if {$len < $minLength} {  
    puts "Length value $len is less than the recommended Min Length  
$minLength"  
    set lenmin [iPDK_sciToEng $minLength]  
    iPDK_setParamValue l $lenmin $inst 0  
}  
if {$len > $maxLength} {  
    puts "Length value $len is greater than the recommended Max Length $maxLen  
gth"  
    set lenmax [iPDK_sciToEng $maxLength]  
    iPDK_setParamValue l $lenmax $inst 0  
}  
}
```

Note:

Commands that start with the prefix `iPDK_` are Custom Compiler built-in commands that ensure that the callbacks are interoperable with other EDA tools.

To assign a callback procedure to a parameter at the cell level:

1. In the Custom Compiler Parameter Definition Editor tool, select a library from the **Library** drop-down list, and then select a cell from the **Cell** drop-down list.
2. Navigate and expand the **Parameters** node in the tree view and expand the specific parameter to which you want to assign the callback.
3. Click the **value** field adjacent to the **callback** attribute, and specify the callback procedure name.
4. From the Custom Compiler Parameter Definition Editor, choose **File > Save**.

Alternatively, click  from the toolbar to finish assigning the callback procedure to the parameter.

Caution:

Callback procedures are typically assigned to parameters at the cell level. However, you can also assign callback procedures to parameters at the library level.

Callback procedures assigned to the cell-level parameters take precedence over the callback procedures specified at the library-level.

Copy Simulation Views From the analogLib Into your Library

To copy simulation views from the analogLib into your library:

1. From the **Custom Compiler Console** menu, choose **Tools > Library Manager**.
Alternatively, click  from the toolbar. The Library Manager window is displayed.
2. In the Custom Compiler Library Manager window, select **analogLib** from the **Libraries** list, and select the cells for which you want to copy a simulation view.
3. From the **Custom Compiler Library Manager** menu, choose **Edit > Copy > CellView**. The Copy CellView dialog box is displayed.
4. Select your library from the **Libraries** list in the **Destination** group into which you want to copy the cellview, and click **OK**. The selected cellview from the analogLib library is copied into your library.

4

Creating Back-End Components

Describes how to create back-end components of an Interoperable Process Design Kit (iPDK).

The following back-end components are used for representing the devices at the layout level:

- **PyCells:** Represent the devices placed on the layout.
- **ivpcell views:** Represent the symbol of the device placed on the layout.

This section covers the following topics:

- [Creating Interoperable PCells Using the Python Language](#)
- [Creating ivpcell Views](#)

Creating Interoperable PCells Using the Python Language

PyCells are OpenAccess-based parameterized cells (PCells) that allow the designers to specify a set of input values for a PCell and instantiate that PCell in any OpenAccess design database.

A PyCell supports the following layout design capabilities:

- Auto-abutment of MOSFETs
- Stretch handles for MOSFETs
- DFM options

The Custom Compiler tools support PyCells for all oaDesigns, which includes oaReservedViewType, maskLayout, schematic, and schematicSymbol view types.

This section covers the following topics:

- [Writing the PyCell Code](#)
- [Creating the __init__.py File](#)
- [Creating the PyCell SuperMaster](#)

Writing the PyCell Code

The following is the syntax of a PyCell code construct for a cell:

Syntax:

```
from <synopsysPackage> import (
    List of APIs separated by commas.
)
class <className>(<APIbaseClassName>):
    @classmethod
    def defineParamSpecs():
        ...
    def setupParams():
        ...
    def genLayout():
        ...
```

Refer to the PyCell Studio documentation for details.

Following is a PyCell code example for a diode:

```
#####
#
# diode.py
#
#####

# Import APIs to be used for creating the PyCells
from cni.dlo import (
    Box,
    DloGen,
    GapStyle,
    Layer,
    MultiPath,
    Rect,
    Term,
    TermType,
)

# API class names:
#### class name ### base class name
class DiodeTemplate(DloGen):
```


Chapter 4: Creating Back-End Components

Creating Interoperable PCells Using the Python Language

```

# Start of class methods
@classmethod

# Define parameters and their legal values
def defineParamSpecs( cls, specs):
    specs( "length", 1)
    specs( "width", 1)

# Reads and processes parameter values entered by the user
def setupParams( self, params):
    # Diode W & L
    self.width = params["width"]
    self.length = params["length"]

    # Create Term and Pin for inside and outside terminals
    self.insideTermId = Term( "minus",TermType.INPUT_OUTPUT)
    self.insidePinId = Pin("minus","minus")
    self.outsideTermId = Term("plus",TermType.INPUT_OUTPUT)
    self.outsidePinId = Pin("plus", "plus")

# Creates the geometry of the PyCell
def genLayout( self):
    # Reading technology values from tech file.
    techGetRule = self.tech.getPhysicalRule
    impEncDiff = techGetRule("minExtension",Layer('nplus'),
                             Layer('diff'))
    mlEncCont = techGetRule("minExtension", Layer('ml'),
                             Layer('cont'))
    contMinWidth = techGetRule("minWidth", Layer('cont'))
    contSpacing = techGetRule("minSpacing", Layer('cont'))

    # Drawing main diffusion
    mainDiff = Rect(Layer('diff'),Box(0,0,self.width,
                                       self.length))

    # Drawing implant around main diffusion
    mainImpl = Rect(Layer('nplus'),
                    mainDiff.getBBox().expand(impEncDiff))

    # Drawing metal in diffusion.
    mainMetal = Rect(Layer('ml'),
                     mainDiff.getBBox().expand(-0.1))

    # Filling metal with contacts
    contBox = mainMetal.getBBox().expand(-mlEncCont)
    mainMetal.fillBBoxWithRects(layer = Layer('cont'),
                                box = contBox,
                                width = contMinWidth,
                                height = contMinWidth,
                                spaceX = contSpacing,
                                spaceY = contSpacing,
                                gapStyle = GapStyle.DISTRIBUTE)

```

```
# Assigning pin id to metal layer.
self.insidePinId.addShape(mainMetal)

class np ( DiodeTemplate):
# Diode class implements NDiode.

    length = 1u
    width  = 1u
```

Creating the `__init__.py` File

After you create your PyCells, you need to define them in the `__init__.py` file, so that the PyCells are packaged with the iPDK.

Syntax:

```
import <PyCell_filename>
def definePcells(lib):
    lib.definePcell(<PyCell_filename>.<className>, <cellName>)
    ...
```

Example 4 Sample `__init__.py` file

```
import diode
def definePcells(lib):
    lib.definePcell(diode.np, "np")
```

Creating the PyCell SuperMaster

After you create the `__init__.py` file, you need to create the PyCell SuperMaster.

Synopsys provides the **cngenlib** utility to update your OpenAccess database library with different PyCell definitions, or to update the technology binding for your OpenAccess PyCell library.

Syntax:

```
cngenlib --update
        pkg:<package>
        <libraryName>
        ./<path to library>
```

Argument	Description
<code>--update</code>	Option to update the PyCell library.
<code>pkg</code>	Path to the PyCell code directory.
<code>libraryName</code>	Name of the OpenAccess library that must be updated.

Argument	Description
path to library	Path to the OpenAccess library.

Refer to the PyCell Studio documentation for details.

Example 5 Sample cngenlib command

```
cngenlib --update
         pkg:pyCells
         sampleLib "./Diodepycell/sampleLib"
```

Creating ivpcell Views

An ivpcell view is a replica of symbol view. It can be created with schematic PCell with viewType as maskLayout or copied from symbol view. It is necessary for every cell in the iPDK to have a corresponding ivpcell view and it is used to display symbols in the generated StarRC view.

To copy the symbol view of a cell to an ivpcell view:

1. From the Custom Compiler Console menu, choose **Tools > Library Manager**.
Alternatively, click  from the toolbar. The Library Manager window is displayed.
2. In the Custom Compiler Library Manager window, select the library from the **Libraries** list and then select the cells from the **Cells** list.
3. Right-click on the **symbol** from the **Views** list, and then choose **Copy** from the menu. The Copy CellView dialog box is displayed.
4. Specify the **View Name** as “ivpcell” and click **OK**. The cell view is added and populated in the **Views** list.

To create an ivpcell view using python:

1. An extra OA parameter “mag” is created on PCell for proper visualization of the StarRC view.
2. The mag (Magnitude) parameter can be controlled for each device using device map file.

```
// dev_netlist_model dev_lib dev_cell dev_view pin_1 pin_2 [pin_3 pin_4 ....]
pres analogLib presistor ivpcell PLUS MINUS MAG=1.0
pcap analogLib pcapacitor ivpcell PLUS MINUS MAG=1.0
n08 SAED_PDK_14 nfet ivpcell D G S B MAG=1.5 PROPMAP w=wtot
```

5

Preparing for Physical Verification and Extraction

Describes the files you must create for physical verification and extraction purposes.

The following mapping files are required for physical verification and extraction purposes:

- **Device-Map File:** Used to map the devices extracted by StarRC.
- **Output Layer-Map File:** Used to map the Layout versus Schematic (LVS) runset layers onto the technology file LPPs.

This section covers the following topics:

- [Physical Verification and Extraction Prerequisites](#)
- [Rules for Creating Device Mapping File and Output Layer Mapping File](#)
- [Create Device Mapping File](#)
- [Create Output Layer Mapping File](#)

Physical Verification and Extraction Prerequisites

Following are the prerequisites for physical verification and extraction:

- Obtain the Design Rule Check (DRC) runset from the foundry.
- Obtain the LVS runset from the foundry.
- Obtain the StarRC runset from the foundry.

Rules for Creating Device Mapping File and Output Layer Mapping File

Follow these rules while creating a device mapping file or an output layer mapping file:

- Each mapping relation must appear on a new line.
- Each attribute in a mapping relation must be separated with a space.
- Lines that should be treated as a comment must be preceded by two forward slashes ("//").

Create Device Mapping File

The purpose of a device mapping file is to map the devices extracted by StarRC with the ivpcells that should appear for each device in the starrc view produced after running StarRC.

The device mapping file must contain the following data columns in a table-like structure, where each row defines relationship between the extracted device to the corresponding device in the iPDK, Where the column `devModel` in the below example is the extracted device name and the column `cellName` is the name of the device in the iPDK:

```
//Fields:
//devModel devLib      cellName dev_view  termOrder magAttrib  mapProp
//-----
N      reference90RF    n_4t      ivpcell   D G S B    MAG=1.5    PROPMAP
w=wtot
P      reference90RF    p_4t      ivpcell   D G S B    MAG=1.5    PROPMAP
w=wtot
...
```

Create Output Layer Mapping File

The purpose of an output layer mapping file is to map the LVS runset layers to the technology file layer purpose pairs (LPPs) for use in the starrc view produced.

The output layer mapping file must contain the following data columns in a table-like structure, where each row defines relationship between the LVS runset layer name and the corresponding layer name and layer purpose in the Technology file:

```
//Format:
//LVS_Runset_Layer  Techfile_Layer/LPP  Techfile_Layer/LPP  Techfile_Layer/LPP
//-----
poly               POLY1    drawing            POLY1    net              POLY1    subnode
metal1             METAL1   drawing            METAL1   net              METAL1   subnode
metal2             METAL2   drawing            METAL2   net              METAL2   subnode
...
```

6

Qualifying and Packaging the iPDK

Describes how to qualify and package the Interoperable Process Design Kit (iPDK) for end-use.

The final step in the iPDK development phase is qualifying and packaging the iPDK.

This section covers the following topics:

- [Qualify the iPDK](#)
- [Package the iPDK](#)

Qualify the iPDK

You need to verify the following components to qualify the iPDK for end-use:

- **iCDF and Callbacks Validation:**

Check the iCDF and TCL callbacks by forcing wrong values, and verify that the correct error/warning is produced.

- **Netlist Validation:**

Verify the netlisting information and equations used for parameter calculations.

- **PyCell Validation:**

Place numerous combinations of PyCells for devices and check for rendering failure, DRC violations, and device extraction through Layout versus Schematic (LVS).

Package the iPDK

After you have verified the iPDK, you need to package the contents of the iPDK for end-use.

Synopsys recommends creating a top-level folder that contains the following files and folders:

Sub-folder	Contents
<technology_process>	This folder must contain the iPDK device library.
techfiles	This folder must contain the following files: • display.tcl • techfile.tf • tech layer map file
scripts	• Common Folder - This folder must contain any common procedures. • Callbacks Folder - This folder must contain the callbacks used to validate user inputs and perform parameter range checks.
icv/hercules	• DRC Folder - This folder must contain the DRC Runsets. • LVS Folder - This folder must contain the LVS Runsets.
starcc	This folder must contain the following mapping files: • device-map file • output layer-map file

A

Common and Advanced Operations

Describes the common and advanced operations that can be used to develop an Interoperable Process Design Kit (iPDK).

This section covers the following topics:

- [PyCell Studio for Developing PyCells](#)
- [Using the Custom Compiler Parameter Definition Editor Tool](#)

PyCell Studio for Developing PyCells

The PyCells in the PDK are developed using APIs from PyCell Studio. Before you are able to work with these PCells, you will have to set up the APIs.

To set up APIs:

1. Go to the directory where you installed PyCell Studio.
2. Install and source the `tcshrc/bashrc` scripts available in the quickstart folder in the PyCell Studio installation directory.

Using the Custom Compiler Parameter Definition Editor Tool

Use the Custom Compiler Parameter Definition Editor tool to add or edit iCDF parameters of specific cells in a library or the library itself.

This section describes how to do the following tasks in Custom Compiler:

- [Adding iCDF Parameters](#)
- [Adding simInfo](#)

Adding iCDF Parameters

The Interoperable Component Description Format describes parameters and parameter attributes of library components.

To add an iCDF parameter to a library or cell:

1. Invoke the Custom Compiler Parameter Definition Editor tool in one of the two ways:

From the...	choose...
Custom Compiler Console window	Tools > Parameter Definition Editor from the menu.
Custom Compiler Library Manager window	Edit > Parameter Definitions from the menu.

2. In the Custom Compiler Parameter Definition Editor tool, select the following option:

If you want to...	then...
Add an iCDF parameter to a library	Select a library from the Library drop-down list and click  to define the iCDF parameter.
Add an iCDF parameter to a cell	a. Select a library from the Library drop-down list. b. Select a cell from the Cell drop-down list, and click  to define the iCDF parameter.

3. Specify the attributes of the parameters as per the following table:

Attribute	Function / Description
type	Specifies the data type the Parameter Definition Editor editor uses to manipulate the parameter value.
prompt	The identifier used in the Schematic Editor to identify the parameter.
callback	This field is evaluated whenever an instance parameter using this definition is modified.
units	The international system of units (SI) of measurement for the parameter.
defValue	Default value. If no property exists on the instance, then this is the parameter value.
display	Sets the default visibility of the parameter.
editable	This field indicates if the Property Editor allows direct editing of parameter instance values. This field is typically used when the value of the parameter is meant to be set by another parameter's change callback.
use	This field combined with the "display" field specifies if the parameter should be displayed in the Property Editor.
parseAsPEL	Controls whether instance parameters should use the PEL expression evaluator.

Attribute	Function / Description
parseAsNumber	Determines whether the result of the evaluation should be treated as a number or as a string literal.
StoreDefault	Upon instantiation, copies the defValue as a property on the instance.

Note:

Only “string” type parameters are considered for PEL evaluation. As such the parseAsPEL field only affects parameters of type “string”.

For more information on parameter and substitution expressions, see Parameter Expression Language (PEL) and Substitution Expression Language (SEL) in the *Custom Compiler™ Expression Language Reference Manual*.

Adding simInfo

The simInfo node in the Custom Compiler Parameter Definition Editor tree view lists information commonly used by the parameterized netlist for generating simulator-specific instance and definition properties.

To add a simInfo definition to a cell:

1. In the Custom Compiler Parameter Definition Editor tool, select a library from the **Library** drop-down list, and then select a cell from the **Cell** drop-down list.
2. Navigate and expand the **simInfo** node in the tree view, right-click the **simInfo** node, and choose **Add Netlist Info** from the menu.

A new blank netlist property is added under the simInfo node in the tree view.

3. Click the blank netlist property entry, and select the netlist tool for which you want to specify the properties.
4. Right-click the netlist tool that you chose in step 3, and choose **Add cdf prop** from the menu.

A blank property is added under the chosen netlist node in the tree view.

5. Click the blank property entry, and choose a property from the properties drop-down list:

Property	Description
componentName	Used to specify a default model name.
instParameters	Used by the netlist formatter's for properties which should be written as “name=value” pairs.

Property	Description
namePrefix	Defines the valid first character prefix for a component.
netlistProcedure	Formatting procedure called to write instance lines for the given component.
permuteRule	Used by the Custom Compiler tool to check what terminals may be permuted during modification.
propMapping	Used to specify a mapping from component parameter names to simulator specific property names
termOrder	Used to specify the order of terminals to be used in an instance line.

6. From the Custom Compiler Parameter Definition Editor, choose **File > Save** or click  from the toolbar to save the netlist and property information.

Glossary

analogLib

Symbols provided by the Silicon Integration Initiative (Si2)

Attribute

Information concerning an object in the database. For instance, the point list of a wire, the color of a drawn wire and the origin of an instance are all object attributes

Callback Syntax

The format used for registering callbacks for parameter fields

CDF

Component Description Format. This language defines information about components in a design library, including parameters and their default values, tool-specific information such as the netlisting procedure and instance parameters, terminal/property mapping, and default annotation settings

Component

Element of a library. Components correspond to OpenAccess database cells within libraries. So an “nmos” in the reference library “synAnalogLib” would be considered a component. A component has one or more associated OpenAccess cellViews, such as symbol, hspice, and schematic

Constraint

Design rule to be enforced in the design

ConstraintGroup

Collection of design constraints, it also has name and properties; corresponds to OpenAccess object oaConstraintGroup

DBU (Database Unit)

Database unit - the unit of measurement used for the design database. Database units are integers, to eliminate rounding errors

iPDK

Process Design Kit that can be used with other OpenAccess based layout tools

LE

Layout Editor

LVS

Layout versus Schematic

Mapping

Refers to logical-to-physical object mapping for SDL

Netlist Info

Netlist Info is a generic container for application data stored at the oaCellIDMData level

oaLayer

Object representing layer in OpenAccess database, it has name, number, and properties

oaPurpose

Object used to provide additional semantics for layer geometries in OpenAccess database. A purpose has a type, number, and name. There are several predefined purposes in OpenAccess, but user applications may define their own purposes as much as they need

Parameter

Parameters are metadata associated with instances and vias. Parameters differ from properties in that they can only be associated with oaRef objects and the definition of the parameter cannot be created directly on an instance or via. The definition of parameters is inherited from the CDF data stored in cell (except for standard vias) and library

Parameter Definition

Refers to the meta-data stored at the cell or library level CDF

PCell

Parameterized Cell

PDK

Process Design Kit - A technology-dependent set of libraries, files, and scripts to support designs targeted at a specific foundry process. Loosely includes symbol libraries, physical verification and extraction runsets, and simulation models, although these components may need to be obtained separately

Property

Properties are name/type/value triplets which can be attached to any OpenAccess database object. They are commonly used by the application and users to associate data and flags with design objects

See Parameter/connectDef functions the *Custom Compiler™ Expression Language Reference Manual* to learn more about properties.

Property Editor

The Custom Compiler Parameter, Property and Attribute Editor. This is the main GUI used by design editors to view and modify Parameters, Properties and Attributes

PV

Physical Verification

PyCell

Interoperable PCell implemented in Python using the PyCell Studio API

Runset

Tool-specific command file, usually for physical verification or parasitic extraction (DRC, LVS, LPE)

SDL

Schematic Driven Layout

SE

Schematic Editor

UU

User Unit - the unit of measurement for each object in the design. User Units are generally floating point numbers